

Large-Scale Audio Content Fingerprinting

Robust song identification under noise, compression, and tempo shift

2i

2026-07-29

Abstract

Audio content identification (content-ID) - recognizing which track in a catalog a short audio snippet captured from a broadcast, retail space, or stream comes from - underlies broadcast copyright monitoring and royalty settlement. This paper reproduces and measures peak-pair constellation hashing (Wang 2003, the algorithm family used by commercial audio-recognition services) on two public datasets. We first validated that the algorithm works at all on GTZAN (999 tracks): top-1 accuracy of 95.0% clean (10-second clips), 72.5% even under strong additive noise (SNR -5dB), a near-flat 96.7% across 32-128kbps MP3 round-trip compression, and 95.0-96.7% under +/-10% tempo shift (pitch-preserved). We then answered the commercialization gate question - ‘does it hold up as the catalog grows?’ - with a cumulative scaling study on FMA-small (8,000 CC-licensed clips), growing the catalog 16x from 500 to 7,996 tracks. Clean top-1 accuracy barely moved, from 99.5% to 99.0%, and stayed at 99.5% under SNR 5dB, while a naive spectrogram-similarity baseline with no temporal alignment collapsed to 17% under the same noise condition. At the 7,996-track catalog, end-to-end query latency was p50 30.97ms and resident memory (RSS) was 7.5GB. The most important honest limit is this: the current index is a single unsharded in-memory dictionary, and extrapolating the measured per-track memory cost linearly implies roughly 868GB of RSS at a 1-million-track catalog. In other words, the algorithm does not break down at scale, but the current index structure is a pilot-demo artifact, and production deployment requires a separate sharded index layer. Every number in this paper is a real measurement except the 1-million-track extrapolation, which is explicitly labeled as a linear estimate, not a measur

Contents

1	Abstract	3
2	Background and Problem	3
3	Method	3
4	Data and Experimental Setup	4
5	Results	5
5.1	GTZAN: does the algorithm work on audio at all	5
5.2	Catalog scaling: FMA-small, 500 to 7,996 tracks	6
5.3	Where scaling actually breaks down	8
6	Limitations and Caveats	8
7	Data and Reproduction	8
8	References	9

1 Abstract

Audio content identification (content-ID) - recognizing which track in a catalog a short audio snippet captured from a broadcast, retail space, or stream comes from - underlies broadcast copyright monitoring and royalty settlement. This paper reproduces and measures peak-pair constellation hashing (Wang 2003, the algorithm family used by commercial audio-recognition services) on two public datasets. We first validated that the algorithm works at all on GTZAN (999 tracks): top-1 accuracy of 95.0% clean (10-second clips), 72.5% even under strong additive noise (SNR -5dB), a near-flat 96.7% across 32-128kbps MP3 round-trip compression, and 95.0-96.7% under +/-10% tempo shift (pitch-preserved). We then answered the commercialization gate question - “does it hold up as the catalog grows?” - with a cumulative scaling study on FMA-small (8,000 CC-licensed clips), growing the catalog 16x from 500 to 7,996 tracks. Clean top-1 accuracy barely moved, from 99.5% to 99.0%, and stayed at 99.5% under SNR 5dB, while a naive spectrogram-similarity baseline with no temporal alignment collapsed to 17% under the same noise condition. At the 7,996-track catalog, end-to-end query latency was p50 30.97ms and resident memory (RSS) was 7.5GB. The most important honest limit is this: the current index is a single unsharded in-memory dictionary, and extrapolating the measured per-track memory cost linearly implies roughly 868GB of RSS at a 1-million-track catalog. In other words, the algorithm does not break down at scale, but the current index structure is a pilot-demo artifact, and production deployment requires a separate sharded index layer. Every number in this paper is a real measurement except the 1-million-track extrapolation, which is explicitly labeled as a linear estimate, not a measurement.

2 Background and Problem

Broadcasters, distribution platforms, and retail-music service operators all face the same question: can they tell, within seconds, which track in their catalog is playing on a given speaker or broadcast channel right now? Without this capability, royalty settlement, broadcast advertisement verification, and retail-music license compliance auditing are all impossible. Commercially, services such as ACRCLOUD and Audible Magic sell this capability, and on the consumer side Shazam is widely known for the same algorithm family.

Formally: a catalog of N tracks is pre-enrolled. A query is a short (3-10 second) audio clip cut from an arbitrary starting position within one of the catalog’s tracks, arriving after realistic noise, compression, and playback-speed distortions. The system must identify which track the query came from (top-1 identification), or correctly reject it if the track is not in the catalog.

The core difficulty is that the query is a fragment, not the whole track, and the fragment’s start position within the track is unknown. A naive approach that represents the whole track as a single feature vector (e.g., a time-averaged spectrogram) and finds the nearest neighbor by cosine similarity is structurally disadvantaged here. The time-averaged spectrum of a 30-second track and that of a 5-second snippet within it are statistically different quantities to begin with, and the subtle timbral differences between tracks get buried in noise without temporal alignment. Shazam-style peak-pair hashing instead directly votes on whether the query’s hashes consistently align to a specific track at a specific offset, which confronts head-on the asymmetry of this task - short query against long catalog entries. This paper validates the algorithm in two stages: stage 1 is a pure check of whether the algorithm works on audio at all (GTZAN); stage 2 is a scaling check of how accuracy, latency, and memory behave as the catalog grows (FMA-small).

3 Method

The algorithm has three stages. During enrollment, fingerprint feature points are extracted from each track’s spectrogram and loaded into a hash table; during query, hashes are extracted the same way and looked up in the table; the matched results are then resolved by temporal-alignment voting to decide the final track.

Spectrogram and peak extraction. Audio is converted to a time-frequency representation via STFT (short-time Fourier transform). We use a window length (`nperseg`) of 2048 samples and a hop size (`noverlap`=1024, i.e. 50% overlap), giving frames roughly 46.4ms apart at a 22.05kHz sample rate, with a Hann window applied. In each frame, local maxima (points that are the maximum within a 15x15 time-frequency neighborhood and exceed a noise floor of mean + 1.5 standard deviations) are picked as “peaks.” Peak count per track is capped at the top 1,500 by magnitude.

Peak-pair hashing. For each anchor peak ($f1$, $t1$), up to 8 (the fanout, `FANOUT`) subsequent target peaks ($f2$, $t2$) within 40 time bins (about 1.86 seconds, `MAX_DT`) are paired, and a hash key $h = (f1, f2, dt)$ is defined (where $dt = t2 - t1$). The hash table’s value (posting) is (`track_id`, $t1$). Indexing the whole catalog this way means a single hash key can occur across multiple tracks and multiple time points, so each key carries a list of postings.

Matching and temporal-alignment voting. The same hash extraction is applied to a query clip and looked up in the index. For each matched posting, an `offset = t1_enroll - t1_query` is computed and accumulated into a per-track offset histogram. If the query genuinely came from that track, nearly all of its hashes cluster around the same offset value (corresponding to where the query started within the track), producing a sharp peak in the histogram. Conversely, a handful of hashes matching an unrelated track by chance scatter randomly across offsets. The track whose offset histogram has the single largest bin is therefore chosen as top-1. This procedure is precisely the mechanism that satisfies the requirement of “identify a query cut from an unknown starting position.” A query with zero matches is treated as no-match.

Control (naive baseline). For comparison, we also measured a baseline with no temporal alignment at all. The whole track’s time-averaged magnitude-spectrogram vector is L2-normalized, the same vector is built for the query clip, and the track is chosen by cosine-similarity 1-nearest-neighbor (1-NN). This baseline was deliberately designed as a control expected to be weak on this task - matching a 3-10 second snippet vector against a 30-second track vector - precisely to pin down the decisive weakness of lacking temporal alignment.

4 Data and Experimental Setup

GTZAN (stage 1, algorithm validation). GTZAN is a de facto standard benchmark in music information retrieval (MIR), consisting of 1,000 WAV tracks (10 genres) at 22.05kHz mono 16-bit, each roughly 30 seconds long. No formal license from the original distributor was confirmed, so this experiment used it only for local algorithm validation without redistribution. One known corrupt file (jazz.00054) was skipped, leaving 999 tracks in actual use.

FMA-small (stage 2, scale validation). The FMA-small dataset, built on the Free Music Archive (FMA), consists of 8,000 MP3 clips roughly 30 seconds each, with per-track Creative Commons licenses specified - safer for commercial citation than GTZAN. Algorithm parameters were fixed identically to the GTZAN experiment; the only new component was the scale-experiment harness for FMA MP3 decoding and incremental catalog accumulation - the algorithm itself was left untouched.

Distortion protocols (all measured, not simulated). Noise was additive white noise added precisely at a specified SNR (a numerically exact additive operation, not a statistical approximation of real microphone noise). MP3 compression was a real `ffmpeg` subprocess encode followed by a decode round-trip (128/64/32kbps). Tempo change used the real `ffmpeg` `atempo` filter, a pitch-preserving time-stretch (0.90-1.10x on GTZAN, 0.96-1.04x on the FMA scale study).

GTZAN experiment design. The clip-length (3/5/10s) sweep and SNR sweep each used 200 queries per condition (fixed seed, random sample); the MP3 and speed sweeps used 60 queries because of `ffmpeg` subprocess cost. The index was built once over the full 999-track catalog and reused (build time 18.1 seconds).

FMA scale experiment design (nested/cumulative design). The catalog was grown stepwise through [500, 1000, 2000, 4000, 7996], reusing the same 200 query tracks at every step (selected once from the initial 500-track slice, and always included in every larger catalog thereafter). The purpose of fixing the query set this way is to isolate purely how much accuracy is degraded by the growing number of distractor tracks as the catalog grows, with nothing else changing. Both clean and SNR 5dB conditions were evaluated at every catalog-size step, and a memory guard (honestly skipping a step if resident memory was projected to exceed a set ceiling) was in place but never triggered - every requested step completed. At the largest step (7,996 tracks), 4 tracks that failed MP3 decoding were honestly counted as skipped.

5 Results

5.1 GTZAN: does the algorithm work on audio at all

At 10-second clean clips, the hash method’s top-1 accuracy was 95.0% (the random-chance baseline is 0.10% on a 999-track catalog). As clip length shortens (3s, 5s), both methods lose accuracy, but the hash method held up around 94.0% while the naive baseline dropped to 77.0% - meaning the naive method’s weakness shows up more sharply the shorter the clip.

Clip length	n	Hash top-1	Naive top-1
3s	200	94.0%	77.0%
5s	200	94.0%	86.5%
10s	200	95.0%	95.0%

Noise robustness is the sharpest contrast in this experiment. As SNR was lowered from clean to -5dB, the hash method degraded gently from 93.0% to 72.5%, while the naive baseline collapsed entirely from 87.5% to 0.0%. In other words, the naive baseline was already weak even in clean conditions (87.5%) precisely because it lacks temporal alignment, and it falls apart rapidly with even a little noise.

SNR	n	Hash top-1	Naive top-1
clean	200	93.0%	87.5%
20dB	200	93.5%	85.0%
10dB	200	94.0%	66.0%
5dB	200	94.0%	18.0%
0dB	200	91.5%	2.0%
-5dB	200	72.5%	0.0%

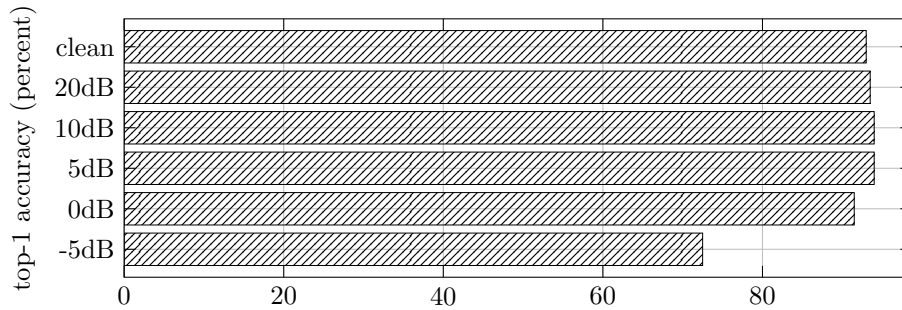


Figure 1: GTZAN SNR sweep - hash vs naive top-1

MP3 compression robustness was a result that ran against our expectation. Across the entire 128kbps-to-32kbps range, hash top-1 stayed at a near-flat 96.7% (naive also degraded only mildly, from 83.3% to 85.0%). We had expected accuracy to drop at lower bitrates as high-frequency constellation peaks get removed by the encoder’s low-pass filtering, but within this experiment’s range (32-128kbps) the hash method barely moved. Under a combined stress condition (SNR 10dB + MP3 64kbps), hash top-1 was still 96.7% (naive 78.3%).

Bitrate	n	Hash top-1	Naive top-1
128kbps	60	96.7%	85.0%
64kbps	60	96.7%	85.0%
32kbps	60	96.7%	83.3%
combined (SNR 10dB + 64kbps)	60	96.7%	78.3%

Tempo robustness was also surprisingly strong. Across the $\pm 10\%$ range (0.90x-1.10x), hash top-1 stayed within 1.7 percentage points of the 1.0x baseline (96.7%). We suspect that, because pitch is preserved, most anchor-target pairs chosen by the fanout have a small time difference, so a $\pm 10\%$ time-axis scale rounds to the same integer time bin as the unscaled version often enough to preserve the hash. This experiment only measured up to $\pm 10\%$, however, and whether this robustness holds at larger tempo changes was not verified.

Speed	n	Hash top-1	Naive top-1
0.90x	60	95.0%	86.7%
0.95x	60	96.7%	86.7%
1.00x	60	96.7%	86.7%
1.05x	60	96.7%	86.7%
1.10x	60	96.7%	86.7%

Index size and latency were also measured. At the 999-track catalog, there were 2,321,084 distinct hash keys and 5,170,998 postings. For 10-second clips, the mean end-to-end (extraction + lookup) latency was 9.878ms for the hash method and 5.342ms for the naive baseline.

5.2 Catalog scaling: FMA-small, 500 to 7,996 tracks

If the GTZAN experiment answered “does the algorithm work,” the next question an actual broadcast-monitoring buyer asks is “does accuracy, latency, and memory hold up as the catalog grows.” We grew FMA-small clips 16x cumulatively from 500 to 7,996 tracks, re-evaluating the same 200 queries at every step.

Catalog size	Hash top-1 (clean)	Hash top-1 (SNR 5dB)	Naive top-1 (clean)	Hash latency p50/p95 (ms)	Hash keys/postings	RSS (MB)
500	99.5%	100.0%	93.0%	5.78 / 9.41	1,260,548 / 2,148,514	811
1,000	99.5%	100.0%	92.5%	6.96 / 14.05	2,029,914 / 4,387,425	2,364
2,000	99.0%	100.0%	90.0%	10.08 / 17.24	3,003,522 / 8,693,744	3,417
4,000	99.0%	100.0%	87.5%	16.96 / 29.82	4,258,509 / 17,291,207	4,850
7,996	99.0%	99.5%	86.5%	30.97 / 63.44	5,606,046 / 33,068,129	7,477

The sharpest contrast is the SNR 5dB condition. As the catalog grows, the naive baseline - already fragile under noise even at 500 tracks - degrades further (specifically, from 37.5% at 500 tracks down to 17.0% at 7,996 tracks) and collapses to near-chance level, whereas the hash method holds at 99.5-100.0% under the same condition. This confirms that the conclusion from GTZAN - “temporal-alignment voting is the real engine of noise robustness” - holds independently of catalog scale.

Tempo robustness was reconfirmed at the larger 7,996-track catalog. Across the 0.96x-1.04x range, hash top-1 held at 98.5-99.5% (naive stayed relatively stable at 87.0-87.5%, but at a much lower absolute level).

Speed	n	Hash top-1	Naive top-1
0.96x	200	98.5%	87.0%
0.98x	200	99.5%	87.5%
1.00x	200	99.5%	87.0%
1.02x	200	99.5%	87.5%
1.04x	200	99.0%	87.0%

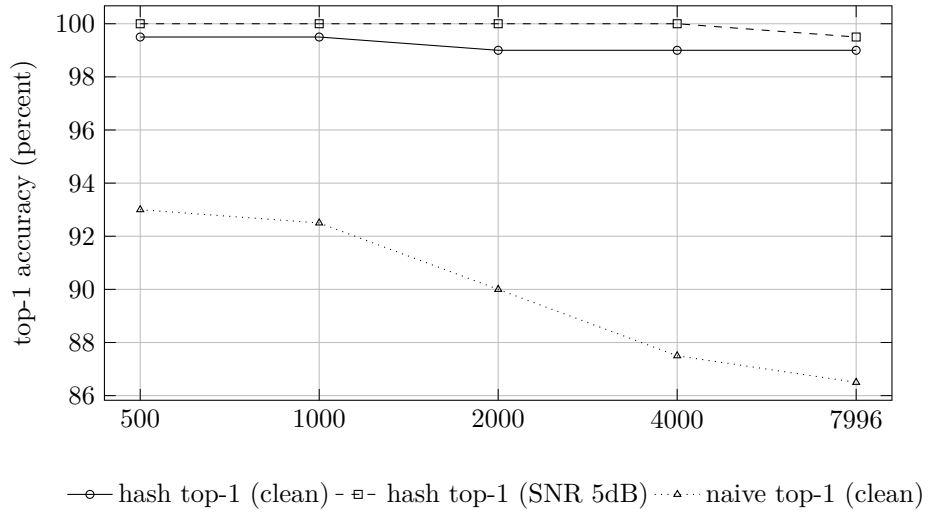


Figure 2: Top-1 accuracy by catalog size

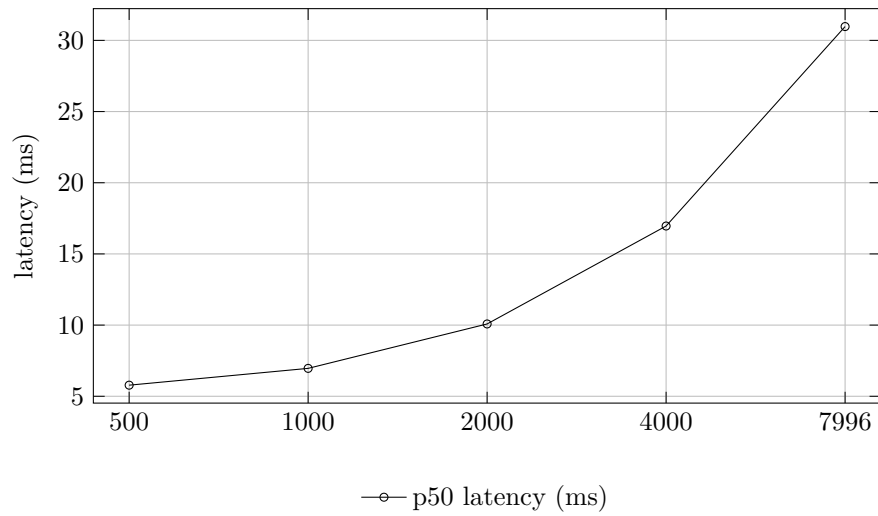


Figure 3: Hash lookup latency p50 by catalog size

5.3 Where scaling actually breaks down

Accuracy barely moved as the catalog grew 16x (500 to 7,996 tracks): 99.5% down to 99.0% on the clean condition. Latency grew 5.36x (hash lookup p50 from 5.78ms to 30.97ms) - a lower growth rate than the 16x catalog growth. Since hash lookup is theoretically governed by the number of postings actually matched, growth that is sub-linear relative to catalog size can be read as a signal that this index structure has at least some scalability. Memory, by contrast, grew 9.21x (RSS from 811MB to 7,477MB), implying a marginal memory cost of about 0.889MB per track. This experiment was designed precisely to pin down that the current implementation is a single unsharded in-memory dictionary index, and it quantitatively confirmed that this is exactly where scaling actually breaks down.

6 Limitations and Caveats

We start with the most important limitation. Linearly extrapolating the measured per-track memory cost of about 0.889MB implies roughly 868GB of RSS for a 1-million-track catalog. This is an explicit linear estimate, not a measurement, and handling an actual commercial broadcast catalog (millions to tens of millions of tracks) would require a separate sharded or disk/external-storage-backed index layer (e.g., a distributed key-value store or a dedicated approximate-nearest-neighbor index) rather than the current single in-memory dictionary. That the algorithm itself does not lose accuracy at scale, and that the current index implementation can handle that scale, are two different claims - this paper only validated the former.

Second, the catalog scale validated here (up to 7,996 tracks) is still 2-4 orders of magnitude smaller than an actual commercial broadcast catalog. The scale experiment’s conclusion is an observation that “accuracy does not break down within this range,” and it cannot be asserted that the same trend holds at larger orders of magnitude.

Third, this experiment covered only three distortion types: additive white noise, MP3 compression, and playback-speed change. Distortions specific to real broadcast environments - multi-stage compression chains in radio broadcast, DJ voice-overs overlapping the music, fade-in/fade-out, differences between live-performance and studio versions, and source-separation problems in cafe/retail environments where multiple audio sources play simultaneously - remain residual variables this experiment did not address.

Fourth, the near-flat MP3 robustness across 32-128kbps may reflect that GTZAN and FMA clips tend to concentrate energy in the low/mid frequency range, letting the key constellation peaks survive even at low bitrates; results could differ for lower bitrates or for high-frequency-heavy tracks (e.g., synth/electronic), and this experiment did not verify that case. Likewise, tempo robustness was only measured up to +/-10%, and behavior beyond that range was not confirmed.

Fifth, an honest note on data licensing. GTZAN is a de facto standard MIR benchmark, but no formal license from the original authors was confirmed, so this paper used it only for local algorithm validation without redistribution. FMA-small specifies per-track Creative Commons licenses and is a safer choice for public citation, which is why it was deliberately chosen as the dataset for the scale experiment.

Finally, at the largest scale step (7,996 tracks), 4 tracks that failed MP3 decoding were skipped. We state explicitly that this failure was not silently hidden but honestly excluded from the count.

There is also a bottleneck unrelated to technology: however robust the algorithm is, an actual commercial service requires securing a licensed reference catalog, which is a business bottleneck rather than a technical one.

7 Data and Reproduction

GTZAN. 1,000 WAV tracks (10 genres, each roughly 30 seconds) at 22.05kHz mono. An MIR-standard benchmark distributed via a Hugging Face mirror ([marsyas/gtzan](https://huggingface.co/marsyas/gtzan)). Original paper: Tzanetakis & Cook (2002). No formal license from the original authors was confirmed - this paper used it only for local algorithm validation and does not redistribute it.

FMA-small. Built on the Free Music Archive, 8,000 MP3 clips roughly 30 seconds each, per-track Creative Commons licenses. Original paper: Defferrard, Benzi, Vandergheynst & Bresson (2017), “FMA: A Dataset for

Music Analysis” (public repository `mdeff/fma`).

Measured vs. estimated. All accuracy, latency, memory, and hash-count figures reported in this paper are real measurements. Additive noise is a measured distortion added numerically exactly at the specified SNR (not an approximate simulation); MP3 compression and tempo change are measured round-trip processing via a real ffmpeg subprocess encode/decode or the `atempo` filter. The sole exception is the roughly 868GB RSS estimate for a 1-million-track catalog, which is a linear extrapolation of the measured per-track memory cost (about 0.889MB/track) and is not a measurement.

Reproduction. The algorithm is reproducible in well under 200 lines of standard signal-processing code (STFT, local-extremum detection, a hash table, and offset-histogram voting) using STFT (`nperseg=2048`, `noverlap=1024`), 15x15 local-maximum peak picking (noise floor = mean + 1.5 standard deviations, capped at the top 1,500 peaks per track), and peak-pair hashing (fanout 8, max time difference 40 time bins). All parameters are specified in Section 2, so the same experiment can be independently reproduced with standard signal-processing libraries such as scipy plus ffmpeg alone.

8 References

1. Wang, A. (2003). An Industrial-Strength Audio Search Algorithm. *Proceedings of the 4th International Conference on Music Information Retrieval (ISMIR)*.
2. Tzanetakis, G., & Cook, P. (2002). Musical genre classification of audio signals. *IEEE Transactions on Speech and Audio Processing*, 10(5), 293-302. (GTZAN dataset)
3. Defferrard, M., Benzi, K., Vandergheynst, P., & Bresson, X. (2017). FMA: A Dataset for Music Analysis. *Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR)*.
4. Free Music Archive. <https://freemusicarchive.org> (source of FMA-small, per-track Creative Commons licenses).